



An Introduction to Neural Networks

Instituto Tecgraf PUC-Rio
Nome: Fernanda Duarte
Orientador: Marcelo Gattass



What is Machine Learning?

A machine learning algorithm is an algorithm that is able to **learn from data**.



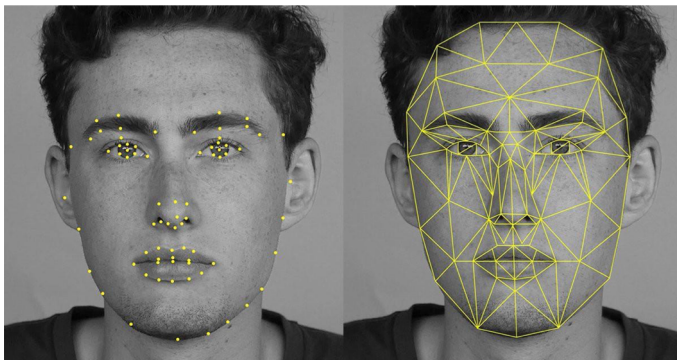
A computer program is said to learn from experience **E** with respect to some class of tasks **T** and performance measure **P**, if its performance at tasks in **T**, as measured by **P**, improves with experience **E**. (Mitchell, 1997)

Applications

Digit Recognition



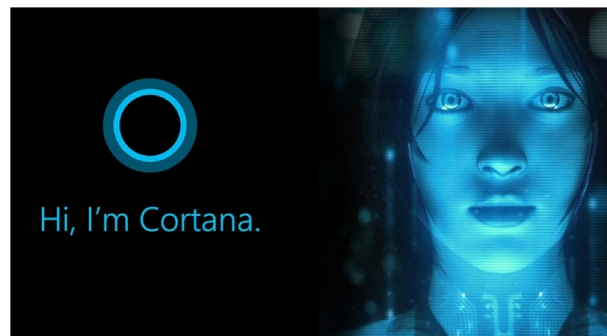
Face Recognition



Recommendation Engines



Virtual assistants (Cortana, Siri etc)



Self-driving vehicles



Surveillance systems





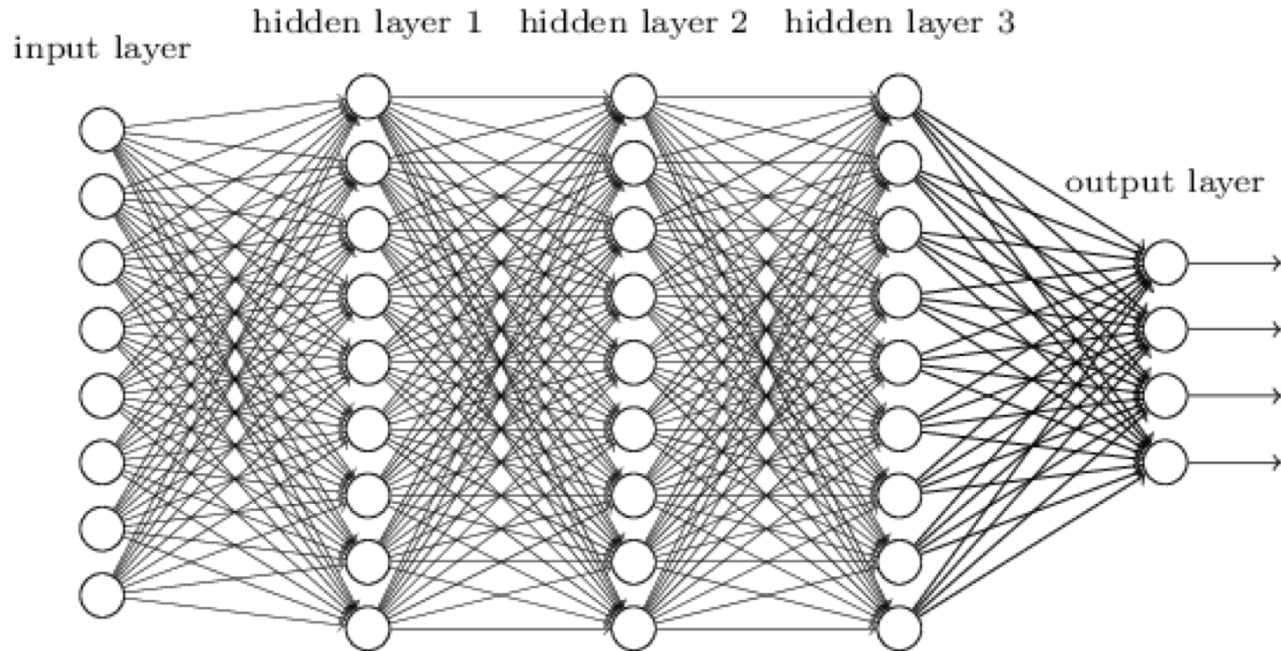
Tasks...

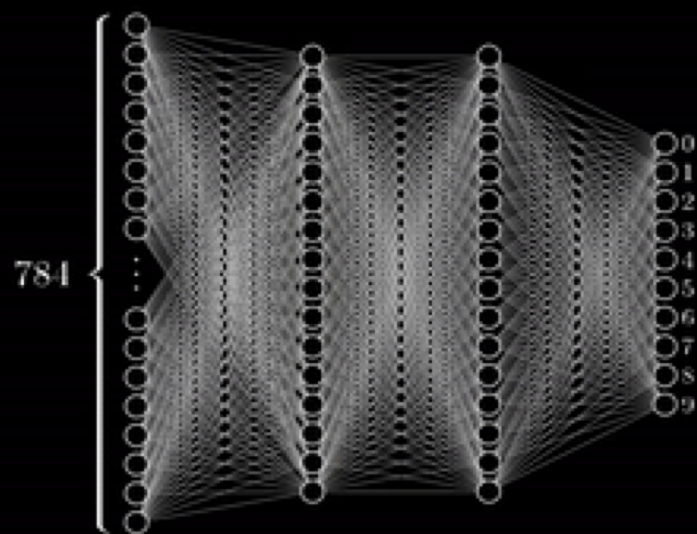
Formal tasks: Playing board games, solving puzzles, mathematical and logic problems → Easier to code!

Expert tasks: Medical diagnosis, engineering, scheduling.

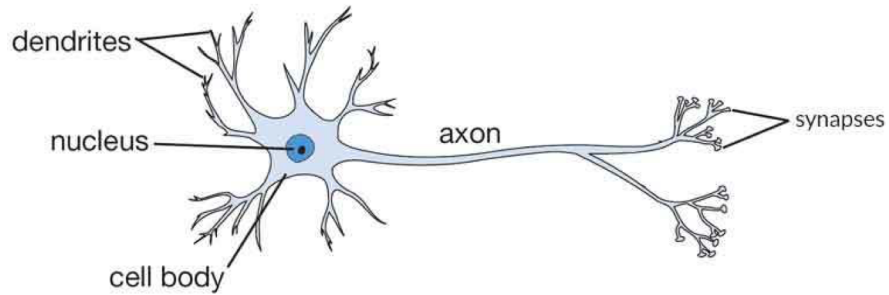
Mundane tasks: Everyday speech, written language, perception, walking, object recognition and manipulation.

Artificial Neural Networks

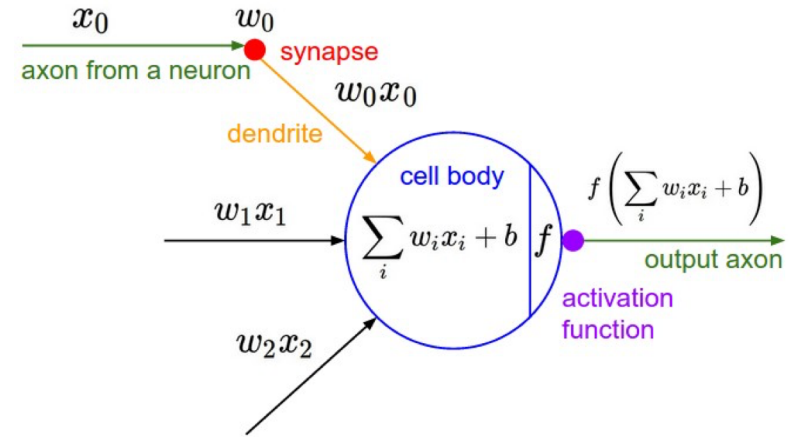




Neuron: biological inspiration for computation



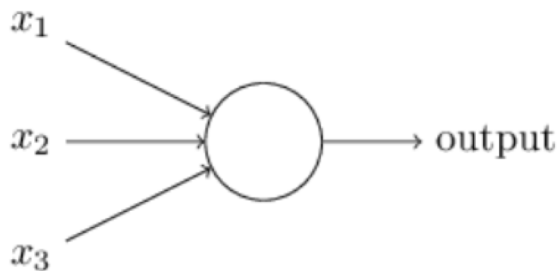
Neuron



Artificial neuron/unit

Perceptron (Frank Rosenblatt, 1957)

- Algorithm for learning a **binary classifier**.
- Only capable of learning **linearly** separable patterns.



$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{threshold} \end{cases}$$

or

$$\text{output} = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$



Feedforward Neural Networks

(or **Deep Feedforward Networks** or **Multilayer Perceptrons (MLP)**) (see “Deep Learning” book, Ian Goodfellow et al.)

- **Multilayer structure** → “sophisticated decision making” (a unit in the second layer can make a decision at a more complex and more abstract level than unit in the first layer) → Learn features directly from the data.
- **Nonlinearity** (extends the kinds of functions that we can represent with our neural network, e.g. XOR function (“exclusive or”))



Feedforward Neural Networks

Goal: Approximate some function f^*

Example $\rightarrow$ A classifier $y = f^*(x)$

input x , category (label) y

A feedforward network defines a mapping $y = f(x; \theta)$ and **learn** the values of the parameter θ that result in the **best function approximation**.

Feedforward: information flows in one direction (input layer $\rightarrow$ output layer)



Why “network”? - Math Intuition:

- Typically represented by composing together many different functions.

Example: functions $f^{(1)}$, $f^{(2)}$, $f^{(3)}$ connected in a chain, to form

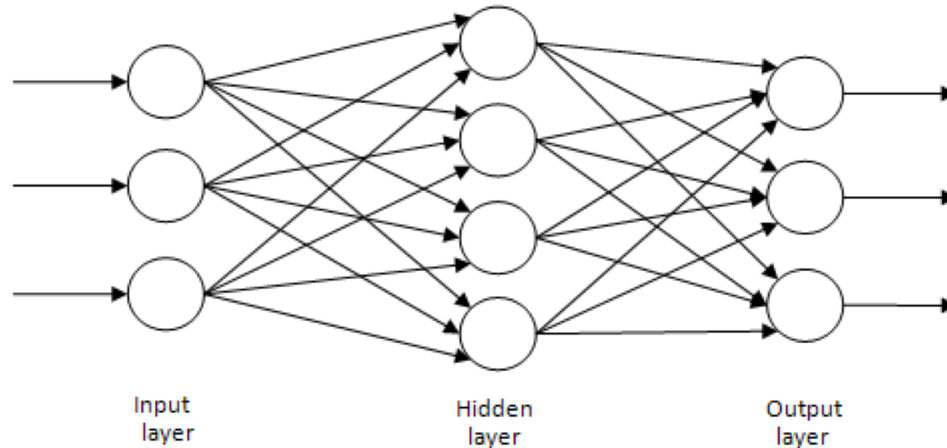
$$f(x) = f^{(3)}(f^{(2)}(f^{(1)}(x)))$$

In this case, $f^{(1)}$ is called the **first hidden layer** of the network, $f^{(2)}$ is called the **second hidden layer**, and the final layer $f^{(3)}$ is called the **output layer**.

- 
- Why *hidden* layer? Behavior not directly specified → learning algorithm must decide how to use those layers to form $f(x)$ that best approximates f^* .
 - Length of the chain gives the depth of the model → **deep learning** (you can “stack” multiple layers)

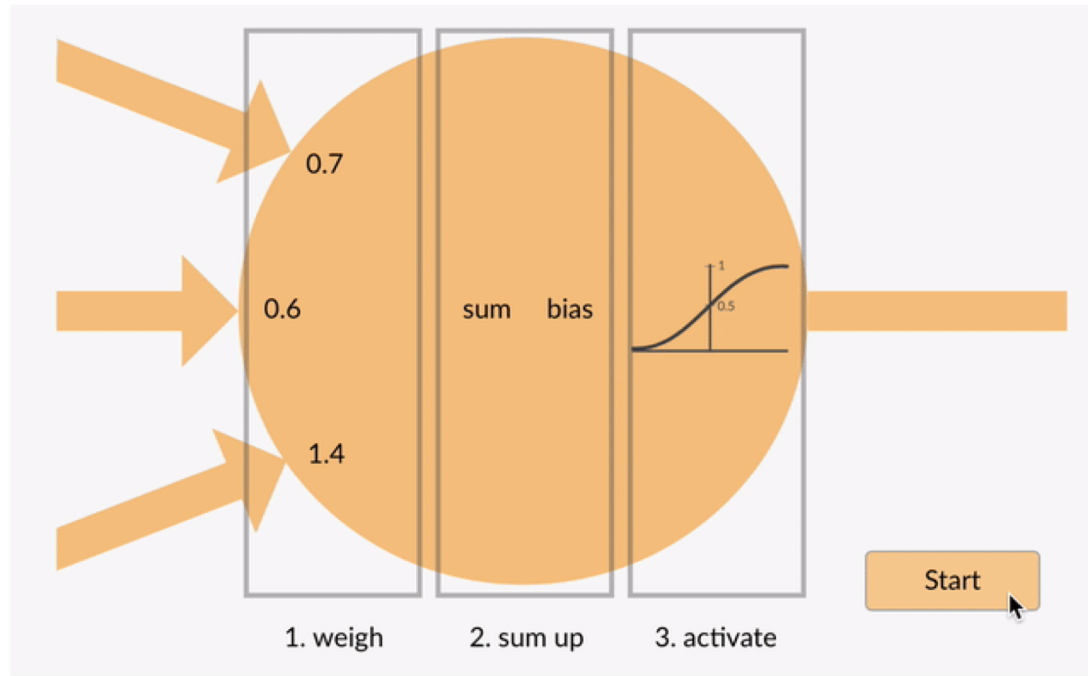
Graph representation of the network

- The feedforward network model is associated with a directed acyclic graph (DAG) describing how the functions are composed together.



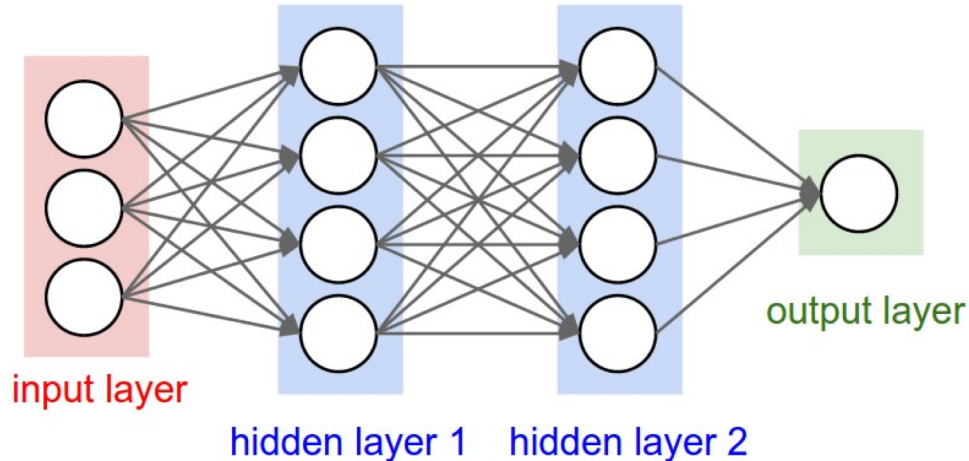
An architectural graph of multi-layer perceptron neural network

Artificial neuron



Fully-connected layers

- Neurons between two adjacent layers are fully pairwise connected, but neurons within a single layer share no connections.



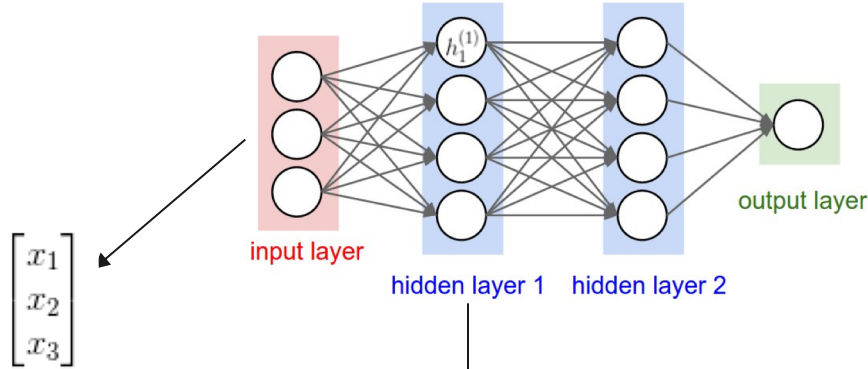


Feedforward computation

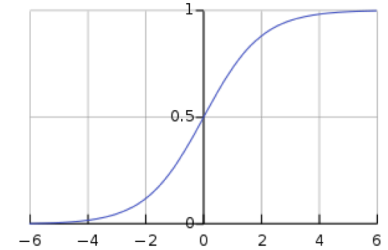
- The abstraction of a layer has the nice property that it allows us to use efficient vectorized code (e.g. matrix multiplies).
- Think of each hidden layer as a vector, where each value represents a neuron/unit.
- Repeated matrix multiplications interwoven with **activation function**.

↘ Nonlinear!

Feedforward computation



Example of activation function: Sigmoid

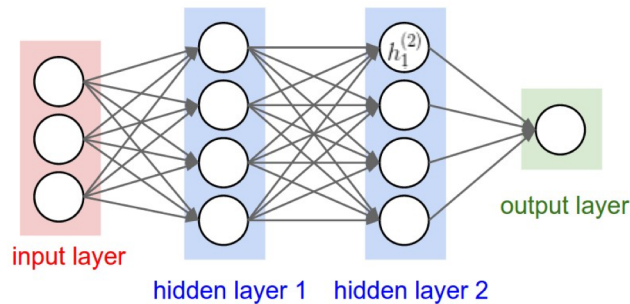


$$\sigma(x) = \frac{1}{1 + e^x}$$

$$H^{(1)} = \sigma(W^{(1)}x + b^{(1)}) = \sigma \left(\begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} & w_{33}^{(1)} \\ w_{41}^{(1)} & w_{42}^{(1)} & w_{43}^{(1)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} \\ b_3^{(1)} \\ b_4^{(1)} \end{bmatrix} \right) = \begin{bmatrix} h_1^{(1)} \\ h_2^{(1)} \\ h_3^{(1)} \\ h_4^{(1)} \end{bmatrix}$$

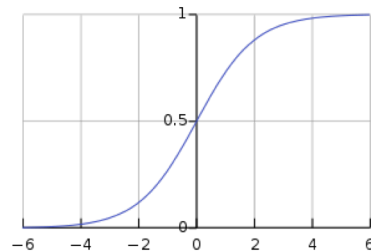
$$h_1^{(1)} = \frac{1}{1 + e^{(w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + w_{13}^{(1)}x_3 + b_1^{(1)})}}$$

Feedforward computation



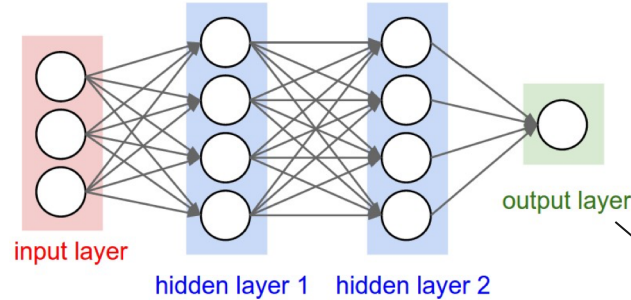
$$H^{(2)} = \sigma(W^{(2)} H^{(1)} + b^{(2)}) = \begin{bmatrix} h_1^{(2)} \\ h_2^{(2)} \\ h_3^{(2)} \\ h_4^{(2)} \end{bmatrix}$$

Example of activation function: Sigmoid

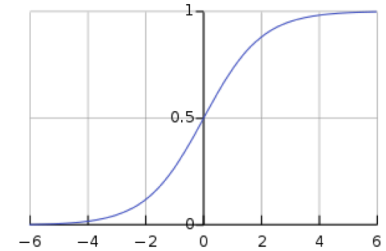


$$\sigma(x) = \frac{1}{1 + e^x}$$

Feedforward computation



Example of activation function: Sigmoid

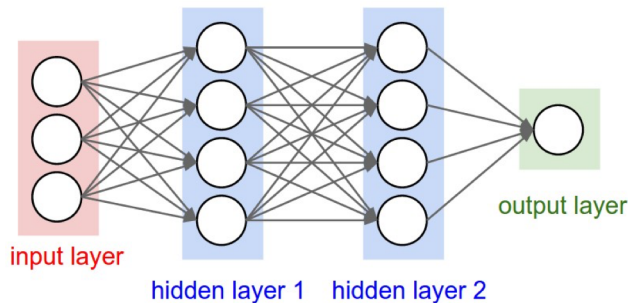


$$\sigma(x) = \frac{1}{1 + e^x}$$

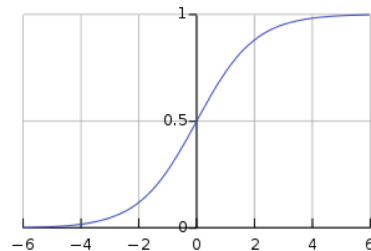
$$\text{Output} = \sigma(W^{(3)}H^{(2)} + b^{(3)}) = \hat{y}$$

Obs.: The output layer neurons most commonly have a different activation function → e.g. softmax for class scores (classification), linear functions for real-valued target (regression), etc.

Feedforward computation



Example of activation function: Sigmoid



$$\sigma(x) = \frac{1}{1 + e^x}$$

$W^{(1)}, W^{(2)}, W^{(3)}, b^{(1)}, b^{(2)}, b^{(3)}$
are the learnable parameters
of the network!

```
# forward-pass of a 3-layer neural network:  
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)  
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)  
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)  
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)  
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```




What about learning?

Optimization: find the parameters that minimizes the **cost function** (or loss function).

A loss function $\mathbf{C}$ is a measure of how wrong the model is in terms of its ability to estimate the relationship between $\mathbf{x}$ and $\mathbf{y}$, i.e., $\mathbf{y} = \mathbf{f}^*(\mathbf{x})$, with the chosen parameters. (e.g. Mean Squared Error (MSE))

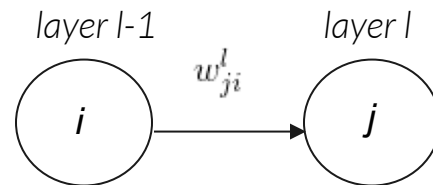
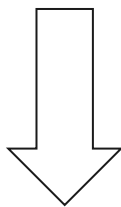


Gradient Descent is a very common optimization algorithm.

Obs.: Training $\rightarrow$ Training + Validation sets
Inference $\rightarrow$ Test set

Gradient Descent

The gradient of a function gives the direction of steepest ascent



The direction of **steepest descent** is the **negative gradient**.

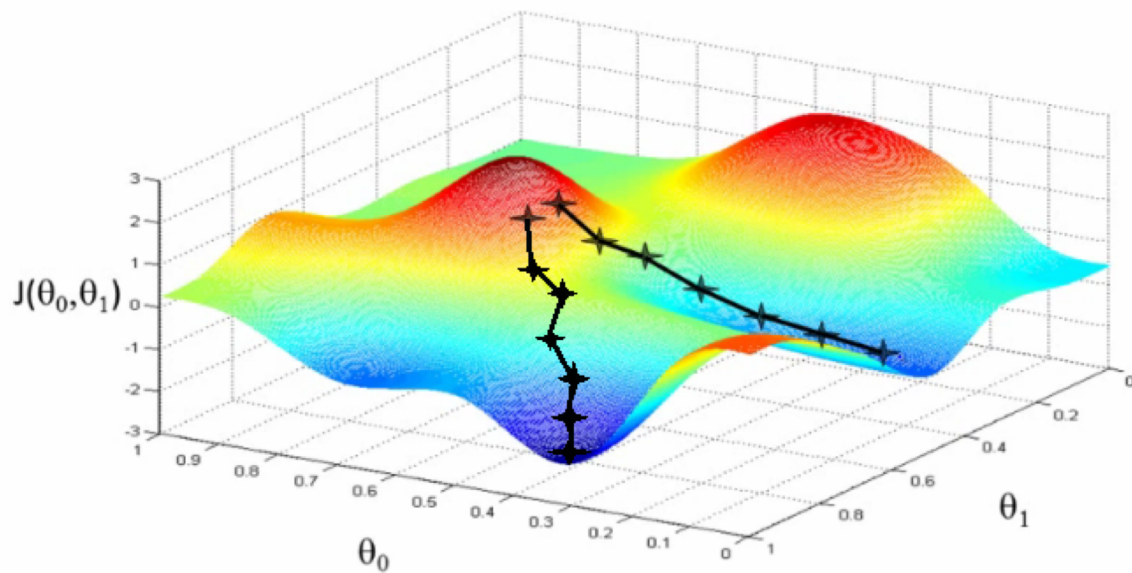
$$w_{ji}^l = w_{ji}^l - \epsilon \nabla_{w_{ji}^l} C = w_{ji}^l - \epsilon \frac{\partial C}{\partial w_{ji}^l}$$

$$b_j^l = b_j^l - \epsilon \nabla_{b_j^l} C = b_j^l - \epsilon \frac{\partial C}{\partial b_j^l}$$

Parameters **update** during learning process.

ϵ is the learning rate (hyperparameter)

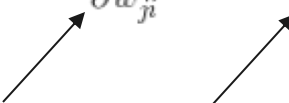
Gradient Descent





Backpropagation

How to compute the gradient of the cost function with respect to the weights $\mathbf{w}$ and biases $\mathbf{b}$ (efficiently)?

$$\frac{\partial C}{\partial w_{ji}^l} \quad \frac{\partial C}{\partial b_j^l}$$


The backprop algorithm gives us detailed insights into how changing the weights and biases changes the overall behaviour of the network.

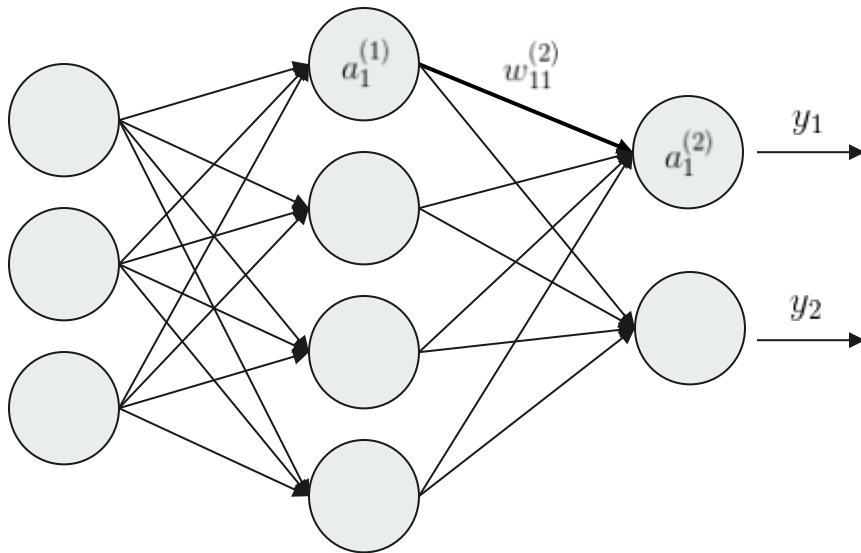
Propagate the **error** backwards.

Main idea: use **chain rule** to compute the gradients.

Backpropagation

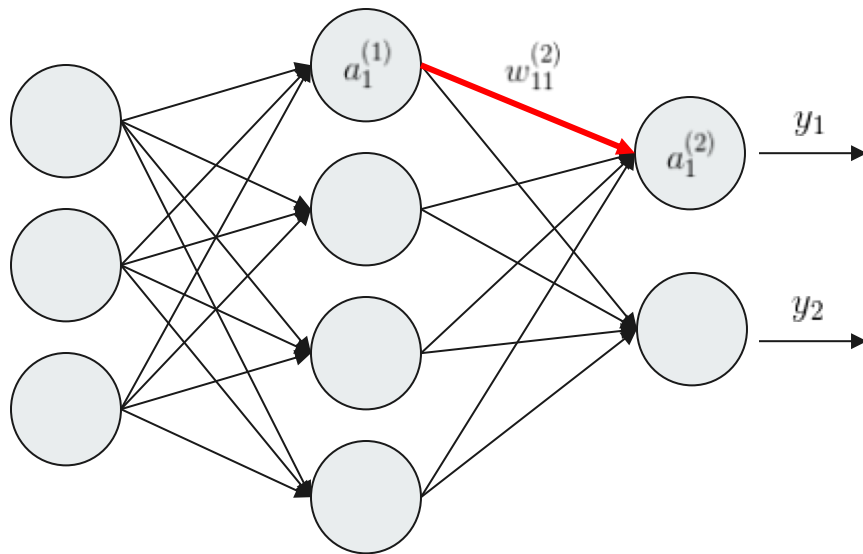
Main idea: use **chain rule** to compute the gradients.

Example: $\nabla_{w_{11}^{(2)}} C_i = \frac{\partial C_i}{\partial w_{11}^{(2)}} = ?$



Backpropagation

Main idea: use **chain rule** to compute the gradients.



Example: $\nabla_{w_{11}^{(2)}} C_i = \frac{\partial C_i}{\partial w_{11}^{(2)}} = ?$

$$C_i = \frac{1}{2} \|a^{(2)} - y\|^2 \quad (\text{Cost function of learning example } i)$$

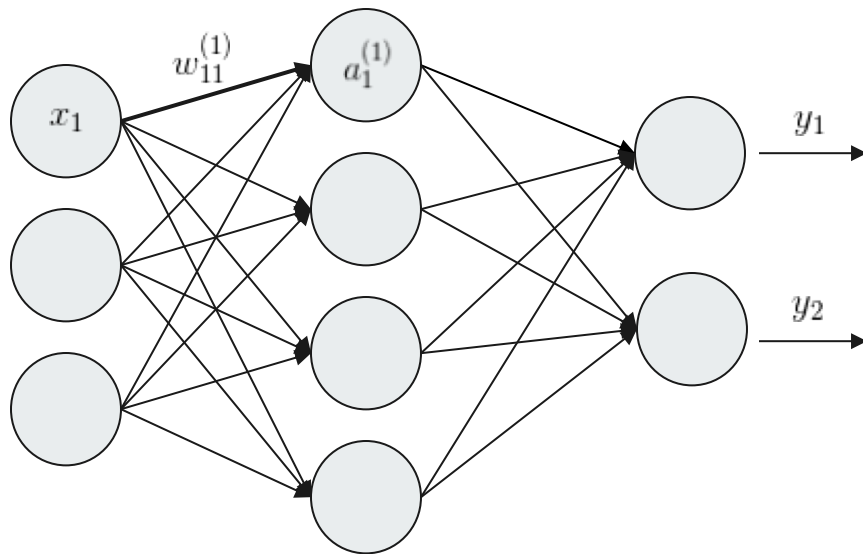
$$a_1^{(2)} = \sigma(\underbrace{x_1 w_{11}^{(2)} + x_2 w_{12}^{(2)} + x_3 w_{13}^{(2)} + x_4 w_{14}^{(2)} + b_1^{(2)}}_{z_1^{(2)}})$$

$$a_1^{(2)} = \sigma(z_1^{(2)})$$

$$\frac{\partial C_i}{\partial w_{11}^{(2)}} = \frac{\partial C_i}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial w_{11}^{(2)}}$$

Backpropagation

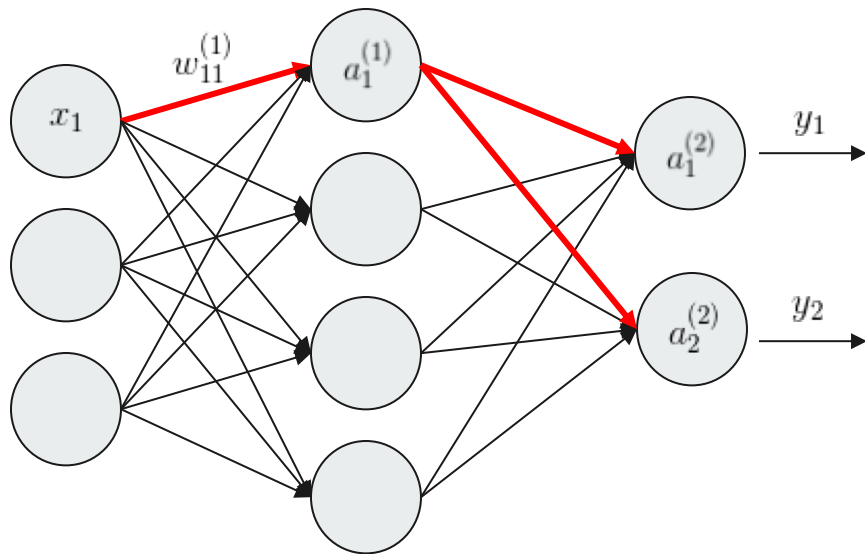
Main idea: use **chain rule** to compute the gradients.



Example: $\nabla_{w_{11}^{(1)}} C_i = \frac{\partial C_i}{\partial w_{11}^{(1)}} = ?$

Backpropagation

Main idea: use **chain rule** to compute the gradients.



Example: $\nabla_{w_{11}^{(1)}} C_i = \frac{\partial C_i}{\partial w_{11}^{(1)}} = ?$

$$C_i = \frac{1}{2} \|a^{(2)} - y\|^2$$

$$\left. \begin{aligned} a_1^{(2)} &= \sigma\left(\sum_{i=1}^4 x_i w_{1i}^{(2)}\right) \\ a_2^{(2)} &= \sigma\left(\sum_{i=1}^4 x_i w_{2i}^{(2)}\right) \end{aligned} \right\} \begin{aligned} a_1^{(2)} &= \sigma(z_1^{(2)}) \\ a_2^{(2)} &= \sigma(z_2^{(2)}) \end{aligned}$$

$$a_1^{(1)} = \sigma\left(\sum_{i=1}^3 x_i w_{1i}^{(1)}\right) = \sigma(z_1^{(1)})$$

$$\frac{\partial C_i}{\partial w_{11}^{(1)}} = \left(\frac{\partial C_i}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \frac{\partial z_1^{(2)}}{\partial a_1^{(1)}} \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} \frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}} \right) + \left(\frac{\partial C_i}{\partial a_2^{(2)}} \frac{\partial a_2^{(2)}}{\partial z_2^{(2)}} \frac{\partial z_2^{(2)}}{\partial a_1^{(1)}} \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} \frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}} \right)$$



Backpropagation

Summary: the equations of backpropagation

$$\delta^L = \nabla_a C \odot \sigma'(z^L) \quad (\text{BP1})$$

$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l) \quad (\text{BP2})$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \quad (\text{BP3})$$

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l \quad (\text{BP4})$$



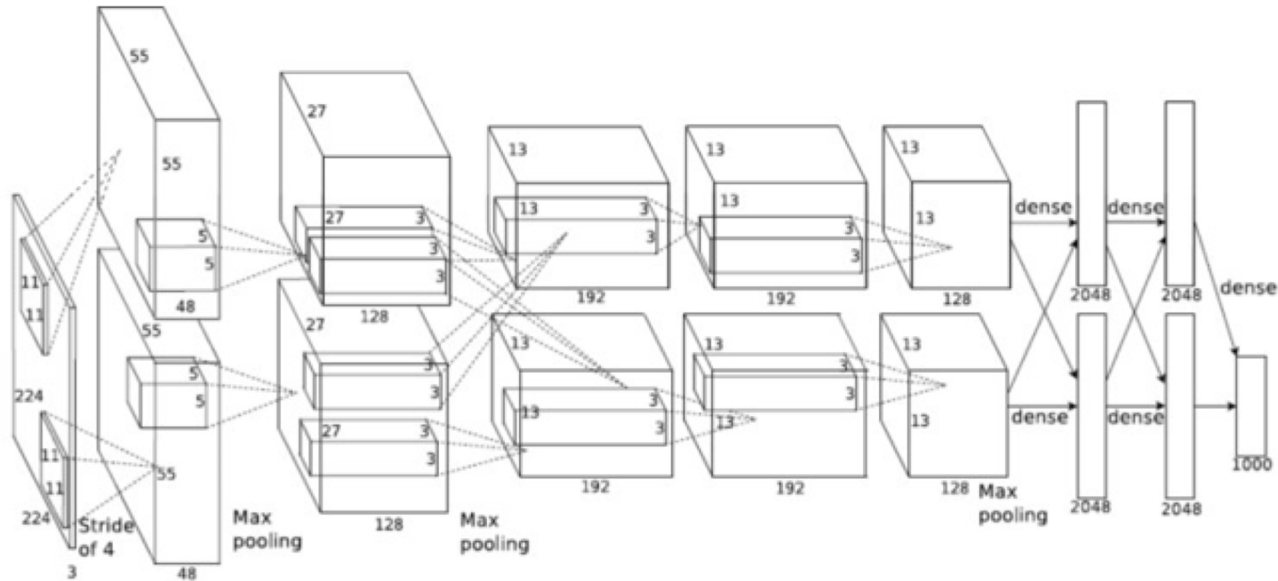
Learning process (summary)

For each learning example i in training set:

- 1 - Feedforward computation;
- 2 - Backpropagation;
- 3 - Weight update.

- 
- **Learning rate (ϵ)**
 - **Regularization (prevent overfitting)**
 - **Epoch**
 - **Activation function alternatives (ReLU, tanh, etc)**
 - **Stochastic Gradient Descent (SGD) → Minibatch**

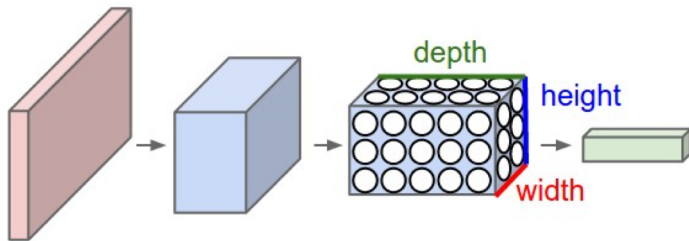
Convolutional Neural Networks (CNN)



(AlexNet)

Convolutional Neural Networks (CNN or ConvNets)

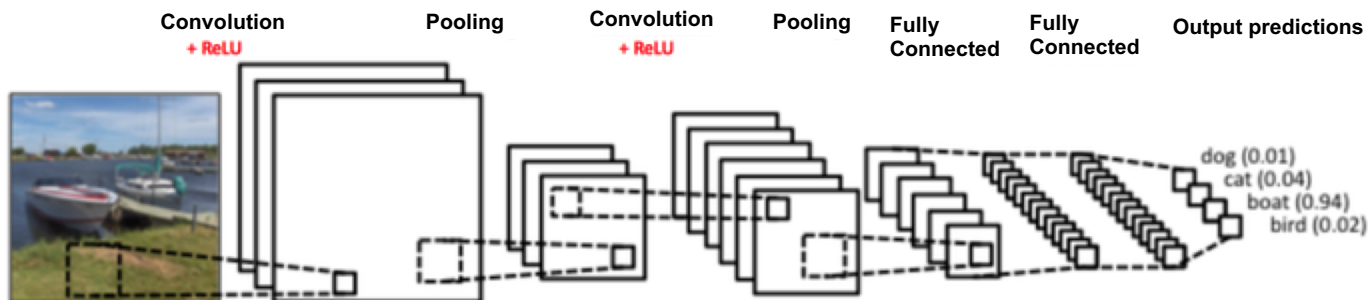
- Useful when the proximity between two data points indicates how related they are (e.g.: pixels in images!) → CNNs preserves spatial structure.
- The neurons in a layer will only be connected to a small region of the layer before it, instead of all of the neurons in a fully-connected manner → less parameters!
- Convolutional Neural Networks take advantage of the fact that the input consists of images and they constrain the architecture in a more sensible way. In particular, unlike a regular Neural Network, the layers of a ConvNet have neurons arranged in 3 dimensions: **width, height, depth**.



Convolutional Neural Networks (CNN or ConvNets)

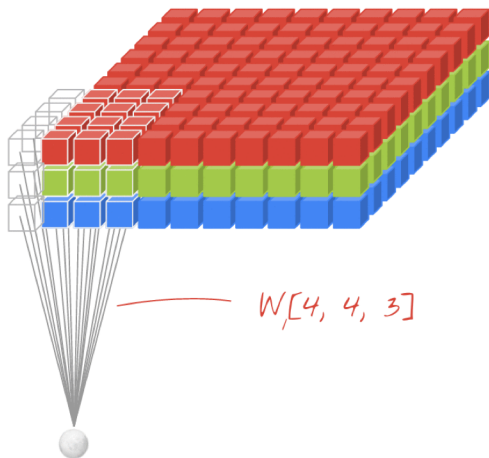
Every layer of a ConvNet transforms one **volume of activations** to another through a differentiable function.

We use three main types of layers to build a basic CNN architecture: **Convolutional Layer**, **Pooling Layer**, and **Fully-Connected Layer**.

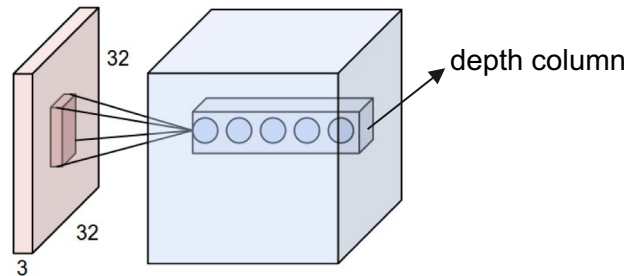


Convolutional Layer

The CONV layer's parameters consist of a set of learnable **filters** → Every filter is small spatially (along width and height), but **extends through the full depth** of the input volume (e.g. 5x5x3 for images with 3 color channels).



Convolutional Layer



Forward pass: We slide (or convolve) each filter across the input volume and compute dot products between the entries of the filter and the input element by element, followed by a nonlinear activation function elementwise.

Every filter produce a 2-dimensional **activation map**. (e.g. if we use 12 filters of dimensions $5 \times 5 \times 3$, the conv layer may have an output volume of $32 \times 32 \times 12$, i.e., 12 activation maps with dimensions 32×32)

Intuitively, the network will learn filters that activate when they see some type of visual feature, such as an edge of some orientation, for example.

zero-padding

Input Volume (+pad 1) (7x7x3)

$x[:, :, 0]$

0	0	0	0	0	0	0
0	0	2	2	0	2	0
0	2	2	0	1	2	0

0	0	2	2	2	1	0
0	0	1	2	1	0	0
0	0	2	2	1	1	0

0	0	0	0	0	0	0
0	0	1	1	1	1	0
0	1	2	2	2	0	0

0	0	1	1	1	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0

0	0	0	0	0	0	0
0	0	2	1	0	2	0
0	2	2	1	1	2	0

0	0	0	2	0	0	0
0	2	1	0	2	1	0
0	2	2	2	1	1	0

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

$w0[:, :, 0]$

-1	-1	-1
-1	0	1
0	0	0

-1	-1	1
-1	-1	0
-1	0	0

1	-1	1
1	1	0
1	0	-1

1	-1	1
1	1	0
1	0	-1

Bias b0 (1x1x1)

$b0[:, :, 0]$

1

Filter W1 (3x3x3)

$w1[:, :, 0]$

-1	-1	1
0	1	-1
1	-1	0

-1	1	0
-1	-1	-1
-1	-1	1

1	-1	0
-1	0	0
0	1	0

1	-1	0
-1	0	0
0	1	0

Bias b1 (1x1x1)

$b1[:, :, 0]$

0

Output Volume (3x3x2)

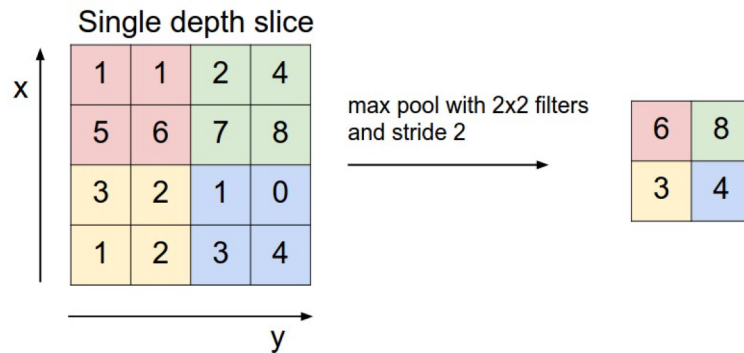
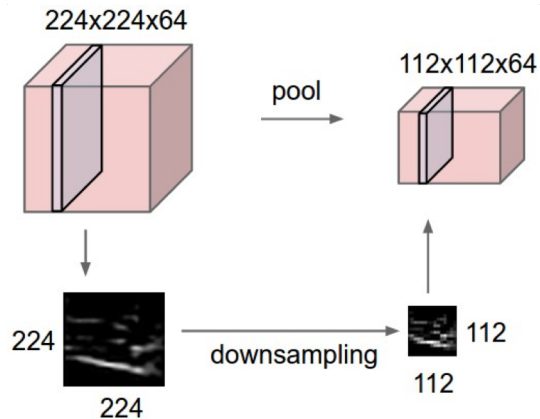
$o[:, :, 0]$

0	-1	-4
-1	-9	-10
0	-5	-2

-2	0	-3
-1	-7	-4
-3	-4	-2

Pooling Layer

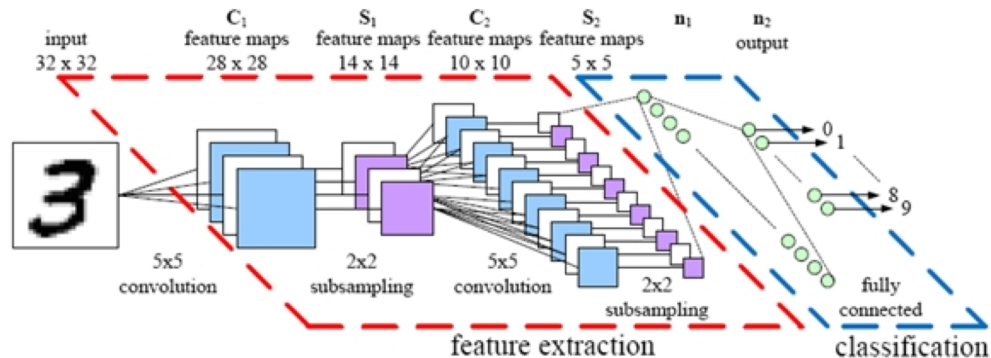
Its function is to progressively **reduce the spatial size** of the representation to reduce the amount of parameters and computation in the network, and hence to also control overfitting.



Fully-Connected Layer

Same as before.

Used to output the final classification scores.





<https://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html>

CNN for Real-Time Object Detection using YOLO ("You Only Look Once")





Trabalho

Construir uma rede neural feedforward com (no mínimo) duas camadas escondidas (fully-connected) para realizar reconhecimento automático de dígitos escritos à mão, utilizando a base de dados MNIST.

<http://yann.lecun.com/exdb/mnist/>

(A descrição do trabalho será enviada por e-mail)



References

<https://www.kdnuggets.com/2018/02/8-neural-network-architectures-machine-learning-researchers-need-learn.html>

<https://www.youtube.com/watch?v=d14TUNcbn1k>

<http://neuralnetworksanddeeplearning.com/chap1.html>

<http://www.deeplearningbook.org/>

<https://www.youtube.com/watch?v=1L0TKZQcUtA>

<https://towardsdatascience.com/machine-learning-fundamentals-via-linear-regression-41a5d11f5220>



References

Backpropagation:

<https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/>

<https://ayearofai.com/rohan-lenny-1-neural-networks-the-backpropagation-algorithm-explained-abf4609d4f9d>

<http://neuralnetworksanddeeplearning.com/chap2.html>

<https://www.youtube.com/watch?v=tleHLnjs5U8>



References

CNNs:

<https://hashrocket.com/blog/posts/a-friendly-introduction-to-convolutional-neural-networks>

<https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/>

http://web.stanford.edu/class/cs231a/lectures/intro_cnn.pdf

<http://cs231n.stanford.edu/>